

AI-ASSIGEMENT-15.4

NAME:- I.NIPUN

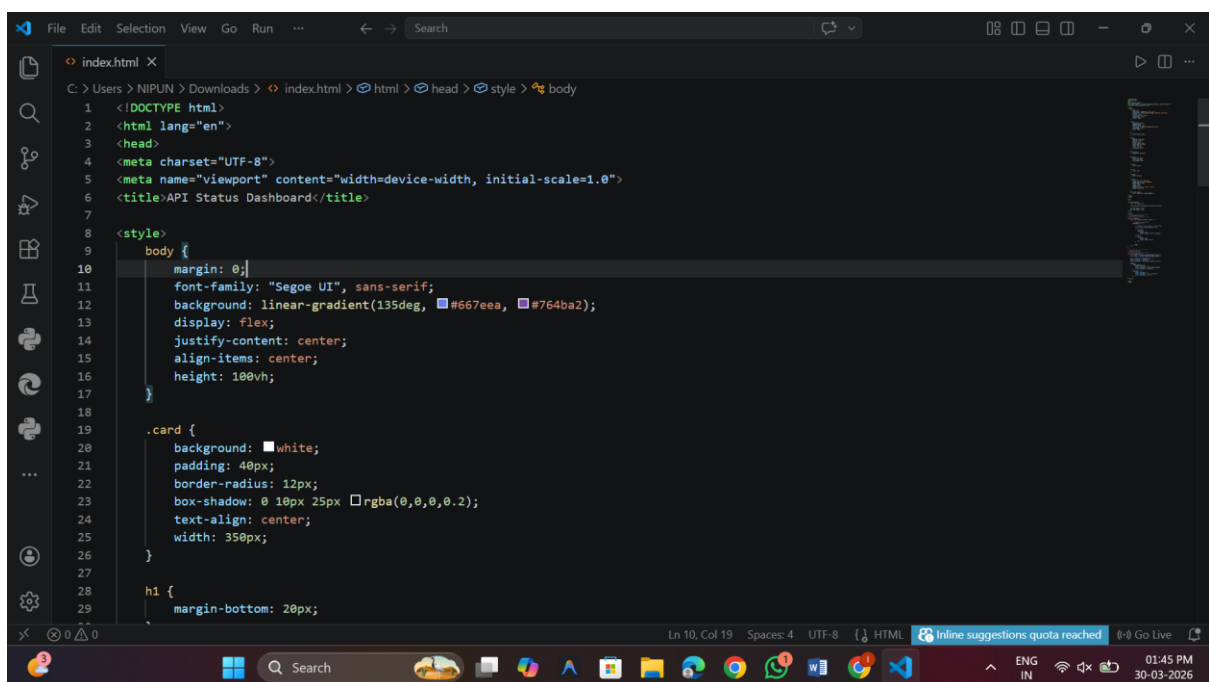
H.T.NO:- 2303A52208

Task 1: Basic API (/status)

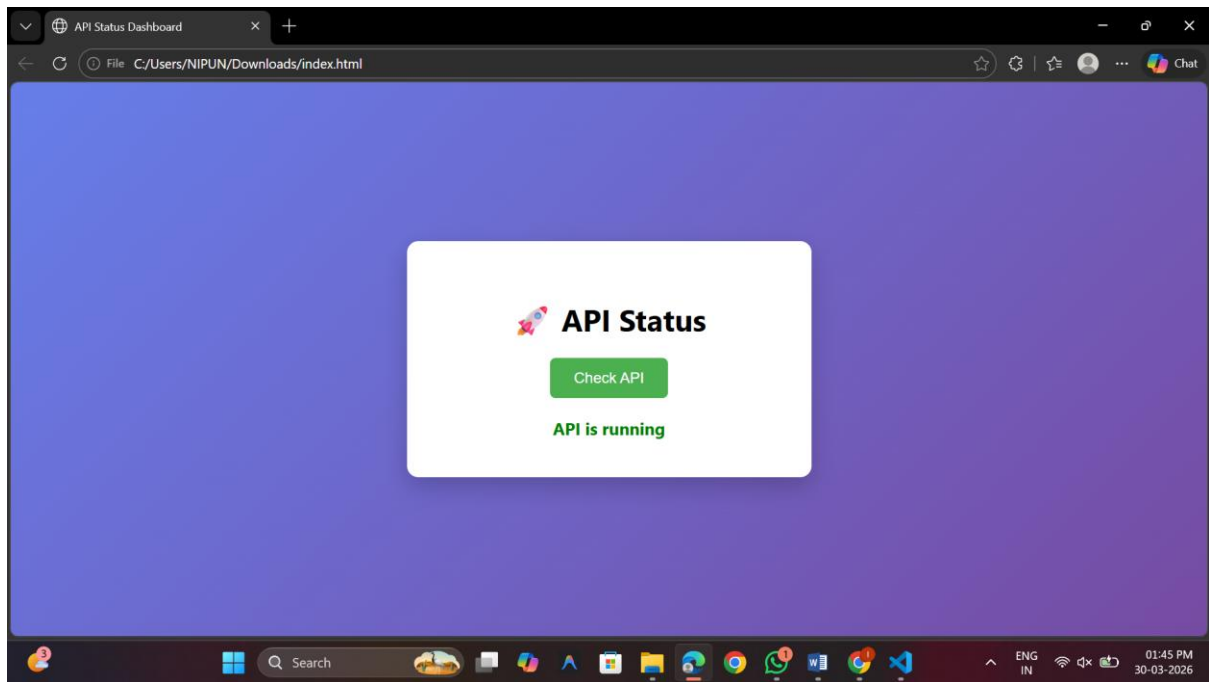
Prompt:

Generate a basic Flask API with a /status endpoint that returns a JSON response {"status": "API is running"}. Include proper structure and comments.

Code:

A screenshot of a code editor window showing an HTML file named 'index.html'. The code includes a DOCTYPE declaration, HTML and head tags, a meta charset, a viewport meta tag, a title 'API Status Dashboard', and a style block. The style block contains CSS for the body (margin: 0, font-family: 'Segoe UI', background: linear-gradient(135deg, #667eea, #764ba2), display: flex, justify-content: center, align-items: center, height: 100vh) and a .card class (background: white, padding: 40px, border-radius: 12px, box-shadow: 0 10px 25px rgba(0,0,0,0.2), text-align: center, width: 350px). There is also a partial h1 tag with margin-bottom: 20px. The editor interface includes a sidebar with icons, a search bar, and a status bar at the bottom showing 'Ln 10, Col 19', 'Spaces: 4', 'UTF-8', and 'HTML'. The Windows taskbar is visible at the bottom with various application icons and the system clock showing 01:45 PM on 30-03-2026.

Output:



Observation:

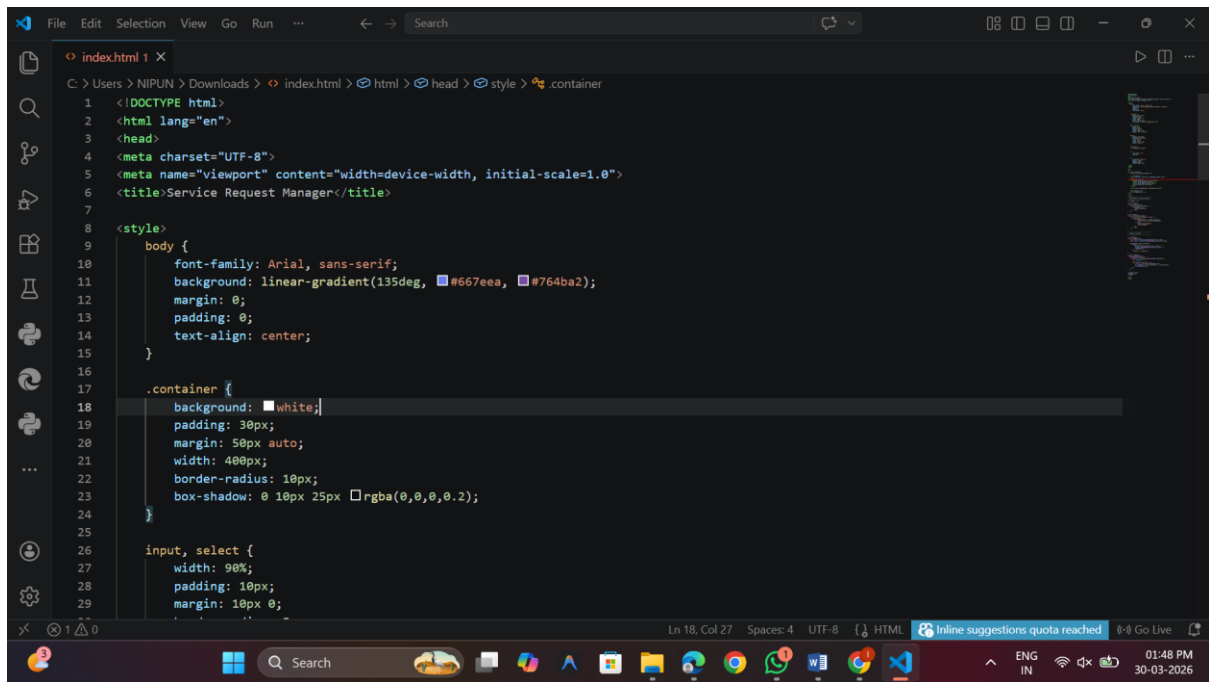
- The API was successfully created using Flask.
- The `/status` endpoint returned a JSON response confirming the API is active.
- The server ran without errors on localhost.
- This task helped in understanding basic API setup and routing.
- AI assistance made it easy to generate boilerplate code quickly.

Task 2: Create API (GET & POST Requests)

Prompt:

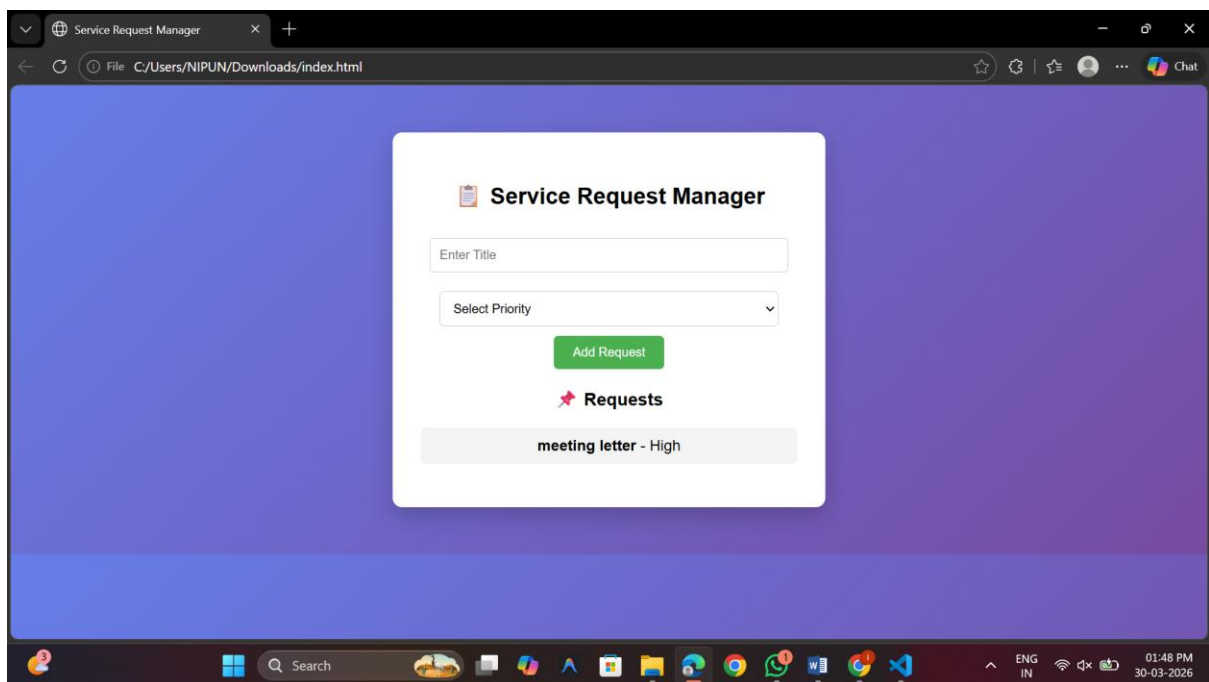
Create a Flask API with GET and POST endpoints for managing service requests stored in memory. Each request should include title and priority. Ensure JSON responses and proper validation.

Code:



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Service Request Manager</title>
7
8 <style>
9   body {
10     font-family: Arial, sans-serif;
11     background: linear-gradient(135deg, #667eea, #764ba2);
12     margin: 0;
13     padding: 0;
14     text-align: center;
15   }
16
17   .container {
18     background: white;
19     padding: 30px;
20     margin: 50px auto;
21     width: 400px;
22     border-radius: 10px;
23     box-shadow: 0 10px 25px rgba(0,0,0,0.2);
24   }
25
26   input, select {
27     width: 90%;
28     padding: 10px;
29     margin: 10px 0;
```

Output:



Observation:

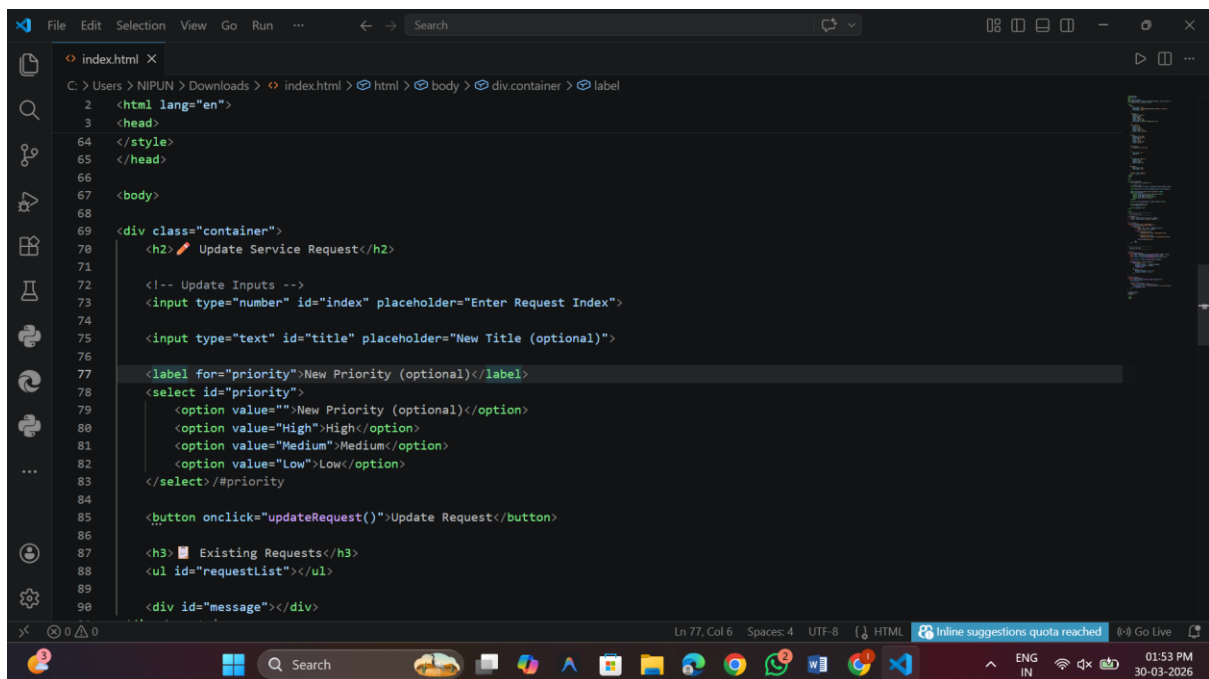
- The GET endpoint returned an empty list initially.
- The POST endpoint successfully added new service requests.
- Data was stored temporarily in memory using a list.
- AI helped structure request handling and JSON formatting.
- The API correctly handled user input and returned expected outputs.

Task 3: Update Resource (PUT Request)

Prompt:

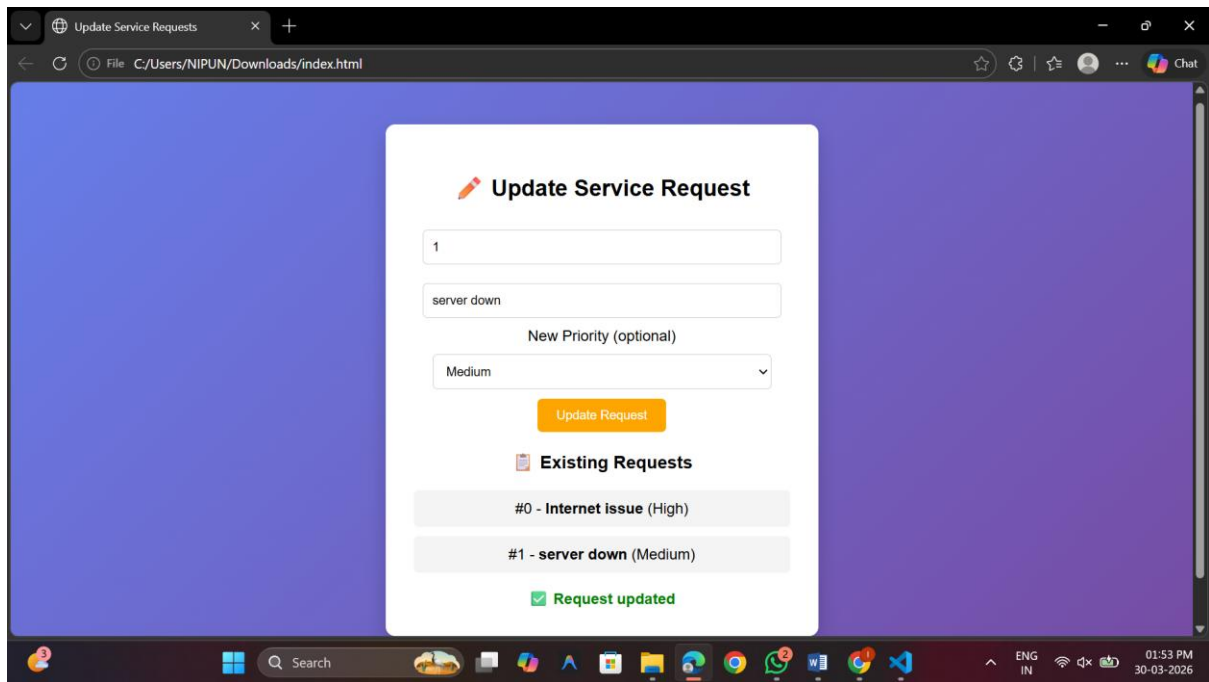
Generate a PUT endpoint in Flask to update a service request by index. Include error handling for invalid indexes and return updated data in JSON format.

Code:



```
index.html X
C:\Users\NIPUN\Downloads> index.html > html > body > div.container > label
2  <html lang="en">
3  <head>
64 </style>
65 </head>
66
67 <body>
68
69 <div class="container">
70   <h2> Update Service Request</h2>
71
72   <!-- Update Inputs -->
73   <input type="number" id="index" placeholder="Enter Request Index">
74
75   <input type="text" id="title" placeholder="New Title (optional)">
76
77   <label for="priority">New Priority (optional)</label>
78   <select id="priority">
79     <option value="">New Priority (optional)</option>
80     <option value="High">High</option>
81     <option value="Medium">Medium</option>
82     <option value="Low">Low</option>
83   </select> /#priority
84
85   <button onclick="updateRequest()">Update Request</button>
86
87   <h3> Existing Requests</h3>
88   <ul id="requestlist"></ul>
89
90   <div id="message"></div>
```

Output:



Observation:

- The PUT endpoint successfully updated existing service requests.
- Proper error handling was implemented for invalid indexes.
- The API returned meaningful responses for both success and failure cases.
- AI improved code readability and structured error handling.
- This task demonstrated how to modify existing resources dynamically.

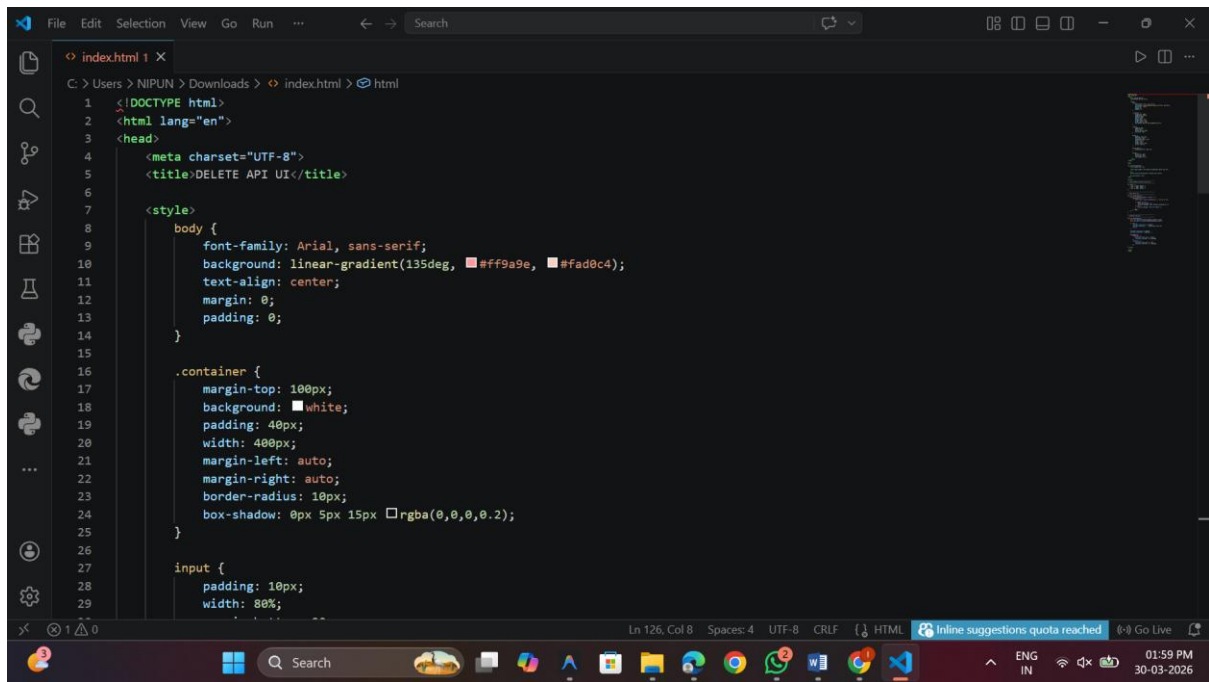
=====

Task 4: Delete Resource (DELETE Request)

Prompt:

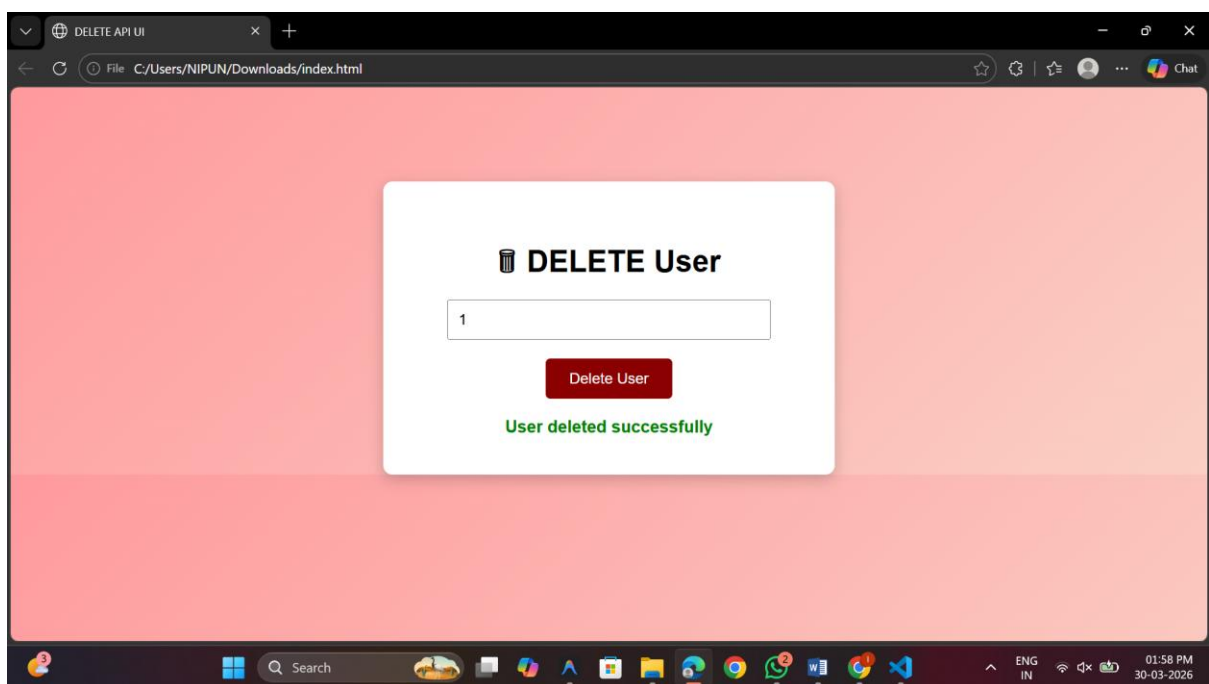
Create a DELETE endpoint in Flask to remove a service request by index. Ensure safe deletion, proper error handling, and return deleted data in JSON format.

Code:



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>DELETE API UI</title>
6
7   <style>
8     body {
9       font-family: Arial, sans-serif;
10      background: linear-gradient(135deg, #ff9a9e, #fad0c4);
11      text-align: center;
12      margin: 0;
13      padding: 0;
14    }
15
16    .container {
17      margin-top: 100px;
18      background: white;
19      padding: 40px;
20      width: 400px;
21      margin-left: auto;
22      margin-right: auto;
23      border-radius: 10px;
24      box-shadow: 0px 5px 15px rgba(0,0,0,0.2);
25    }
26
27    input {
28      padding: 10px;
29      width: 80%;
```

Output:



Observation:

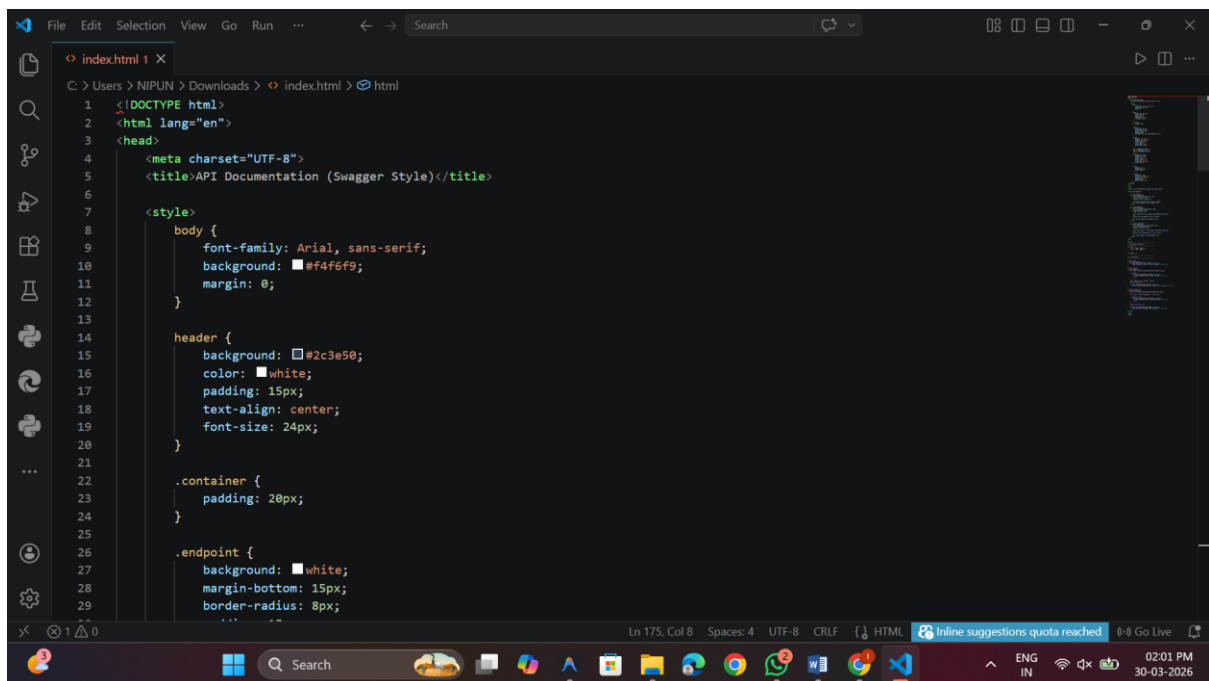
- The DELETE endpoint removed service requests successfully.
- The deleted data was returned as confirmation.
- Invalid index cases were handled without crashing the server.
- AI assistance ensured clean and efficient code.
- This task helped understand safe data removal in APIs.

Task 5: API Documentation (Swagger / Comments)

Prompt:

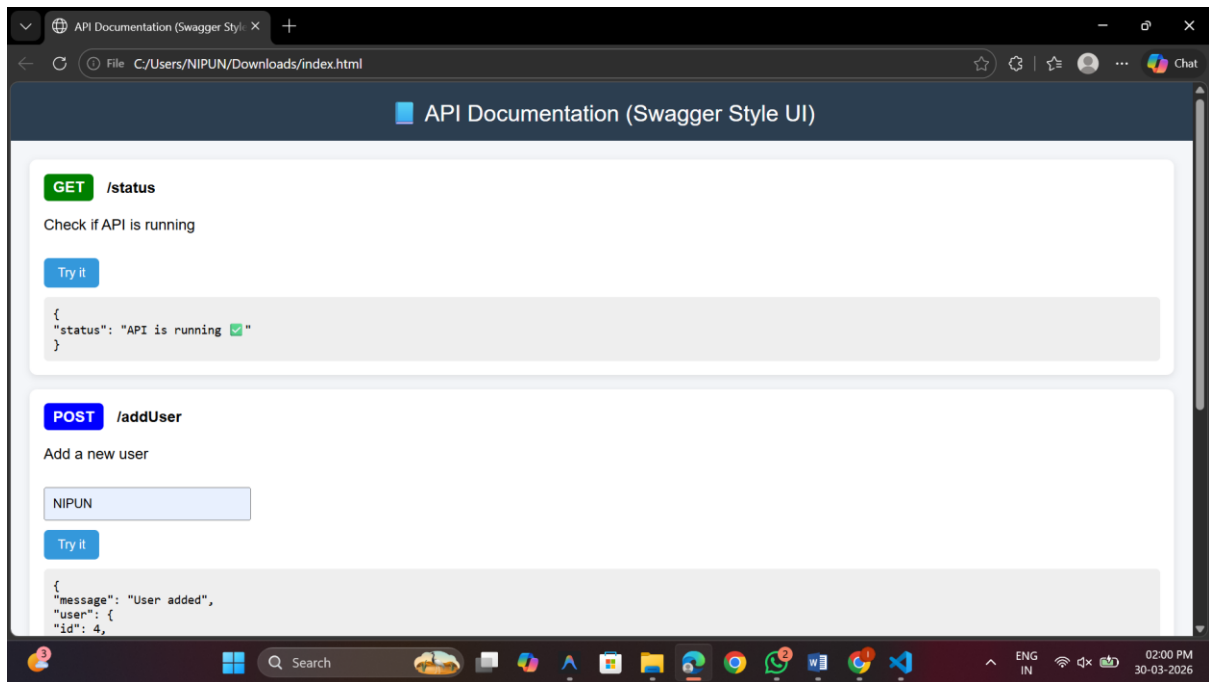
Add detailed docstrings and comments to all Flask API endpoints. Optionally integrate Swagger UI or a simple documentation interface showing endpoints, methods, and sample requests/responses.

Code:

A screenshot of a code editor window showing an HTML file named 'index.html'. The code is written in a dark theme and includes a DOCTYPE declaration, a head section with a meta charset and a title 'API Documentation (Swagger Style)', and a style section. The style section defines a body with a sans-serif font, a dark blue background, and a white header with a 24px font size. A container with 20px padding and an endpoint with a white background and 8px border radius are also defined. The editor's status bar at the bottom shows 'Ln 175, Col 8', 'Spaces: 4', 'UTF-8', 'CRLF', and 'HTML'. The Windows taskbar is visible at the bottom with the time '02:01 PM' and date '30-03-2026'.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>API Documentation (Swagger Style)</title>
6
7   <style>
8     body {
9       font-family: Arial, sans-serif;
10      background: #f4f6f9;
11      margin: 0;
12    }
13
14    header {
15      background: #2c3e50;
16      color: white;
17      padding: 15px;
18      text-align: center;
19      font-size: 24px;
20    }
21
22    .container {
23      padding: 20px;
24    }
25
26    .endpoint {
27      background: white;
28      margin-bottom: 15px;
29      border-radius: 8px;
```

Output:



Observation:

- All endpoints were documented with clear descriptions.
- Inputs, outputs, and methods were properly explained.
- A Swagger-like UI (or structured documentation) improved usability.
- AI helped generate meaningful docstrings automatically.
- Documentation made the API easier to understand and use by other developers.